

“Cvyt”
v0.0.1

Table of Contents

Chapter 1.....	3
What it is this.....	3
Current version of app.....	3
Limitations.....	3
Config file.....	3
Modules.....	3
License.....	3
Goals.....	3
Tags.....	3
Supported platform.....	4
Used packages, dependencies.....	4
Python should be.....	4
Chapter 2.....	5
How to use it.....	5
UI.....	5
Main window – left overview.....	5
Main menu.....	6
Main.....	6
Update.....	6
Module content.....	6
How to.....	6
Config.....	8
Config tree.....	10
Supported type.....	11
Add.....	11
Examples.....	11
Edit.....	12
Remove.....	12
Config tree, value combinations.....	13
Add.....	13
Edit.....	13
Remove.....	14
Help tab.....	14
Info tab.....	14
Chapter 3.....	16
Modules.....	16
Module content structure.....	16
Module import/start.....	16
Update module.....	16
Chapter 4.....	17
Troubleshooting.....	17
Logging.....	17
Everything si broken.....	17
Chapter 5.....	18
Development.....	18
Tests.....	18
Changes, new ideas, cooperation.....	18
Future.....	18
Technical debt & New ideas.....	18

Chapter 1

What it is this

Simple app to help use “*opencv-python*”. This app gives you simple UI and allow you to “*directly*” open you module which is using same dependencies(for now).

Current version of app

- sync
- dynamic import od module

Limitations

Config file

The config file is expected in “*json*” format and it brings few limitations with it

- key has to be string(no number or advanced type)
- value basic options
 - integer
 - float
 - string
- as value you can use
 - dictionary(key can be only a string)
 - list

Modules

You have to specify widget main class in one of three possible ways, otherwise module won’t be imported.

- “class_name” (module_config.json)
- using “__all__” (*widget.py file)
- via “__init__.py” file

License

GPLv3

Goals

Just playing around with Qt(pyside6), with all its pros and cons, my knowledge flaws about this as well.

Tags

#noai #novibecoding #simplepython

Supported platform

This app should be running with Linux environment.

Used packages, dependencies

See “*pyproject.toml*” file, but right now it should be only basic “*python, opencv-python, pyside6*”.

Python should be

≥ 3.12 and < 3.15

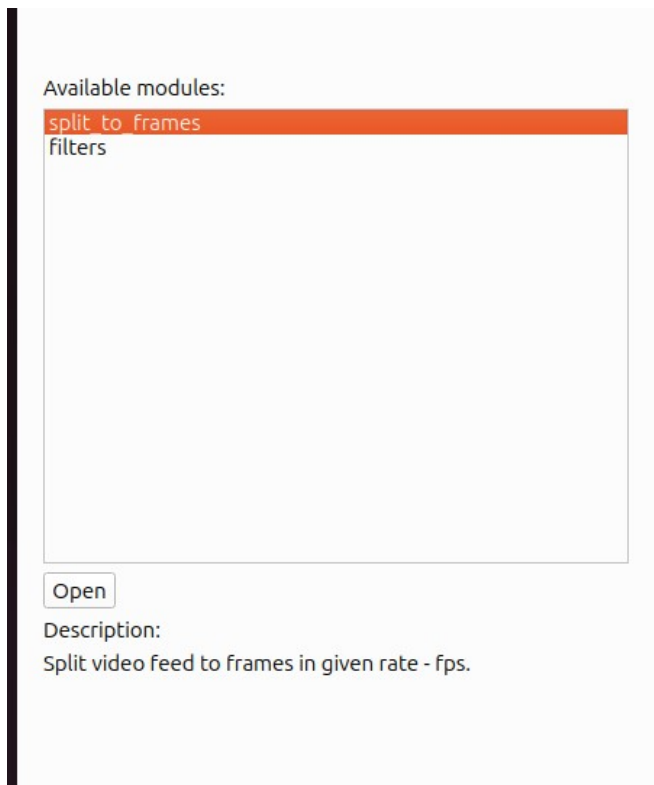
Chapter 2

How to use it

Its up to you how you desire to use this app(.exe, raw python). In simple way possible just start it.

UI

Main window – left overview



This overview contains list of available modules (*“Available modules”*). To get the description for specific module just click on the name of the module(just once) and you should see *“Description”* under the list table.

To open the module in new tab you can do it in two ways.

1. Select the module of you desire(click on the name) and click on button *“Open”*
2. Select the module and do *“double-click”* on that name.

Main menu



Main = contains option “*Exit*” to close the app

Update = opens new tab and you can update/add more modules to be used

Config = opens config tab to “*play*” with config file of this app or eventually of you module’s config file

Info = opens info tab. This tab is going to stay open only even if you close all the other tabs

Help = opens help tab or default browser with “*HOWTO.pdf*” document for this app. If you prefer browser before tab, it can be changed in app config file(default is tab).

Main

This offers you option to “*Exit*” the app.

Update

You can update/add/overwrite modules for “*cvyt*” app. Just prepare just simple archive with structure

```
- modules
|- filters
|- main_widget.py
|- logic.py
|- help.pdf
|- module_config.json
```

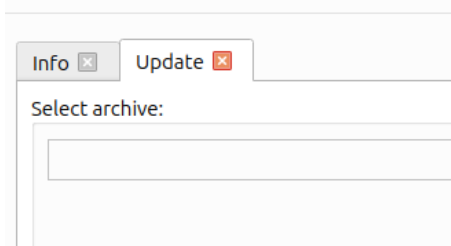
All this in simple “*modules.zip*” archive.

Module content

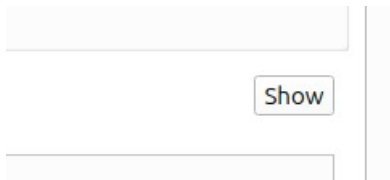
For more “up-to-date” info check “*cvyt-modules*” repository.

How to

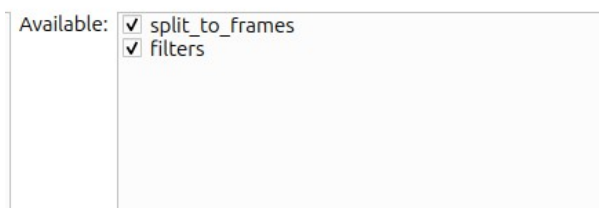
Select the archive via button “*Folder*”.



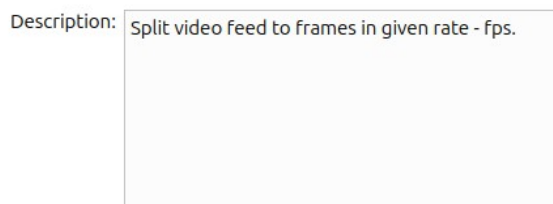
Next step should be to check what it is in the archive(validation of folders in archive). Just click to button “Show”.

A screenshot of a web interface. It features a light gray background with a vertical line on the right side. On the left, there are two horizontal input fields. Between these fields, centered, is a small rectangular button with the text "Show" in a dark gray font.

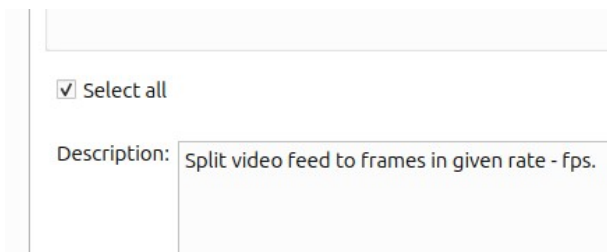
After this you should see something like this.

A screenshot of a web interface showing a list of available modules. The interface has a light gray background with a vertical line on the right. On the left, there is a label "Available:" followed by a list of two items: "split_to_frames" and "filters". Each item has a small checkmark icon to its left, indicating they are selected.

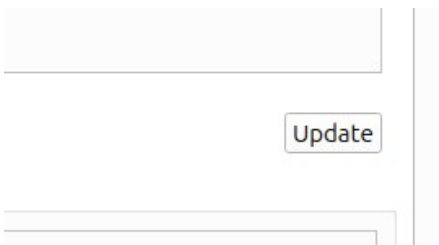
In this example we see archive containing 2 valid modules(“*split_to_frames*”, “*filters*”). In case you want to see simple description for module(its taken from “*module_config.json*” file) just click on the module name.

A screenshot of a web interface showing a description for a module. The interface has a light gray background with a vertical line on the right. On the left, there is a label "Description:" followed by a text box containing the text "Split video feed to frames in given rate - fps."

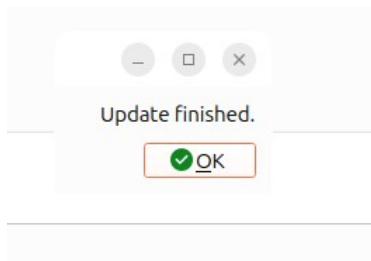
Sometimes you have available archive with a lot of modules and you don’t want set “*checked*” for each module one-by-one. For this situation you can use “*Select all*” option. By default all modules are set-up as “*checked*” for update.

A screenshot of a web interface showing a "Select all" option. The interface has a light gray background with a vertical line on the right. On the left, there is a label "Select all" with a small checkmark icon to its left, indicating it is selected. Below this, there is a label "Description:" followed by a text box containing the text "Split video feed to frames in given rate - fps."

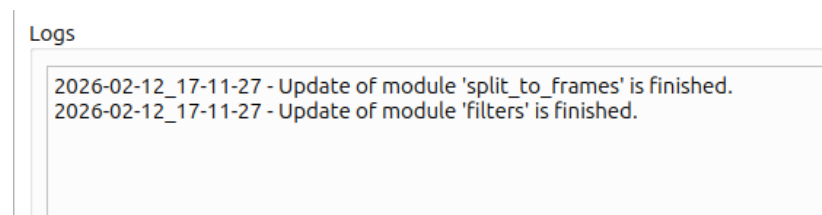
Now is time to start the update itself.

A screenshot of a web interface showing an "Update" button. The interface has a light gray background with a vertical line on the right. On the left, there are two horizontal input fields. Between these fields, centered, is a small rectangular button with the text "Update" in a dark gray font.

After update is done, correctly or incorrectly there should be message.



And for more detailed info check the logs box.



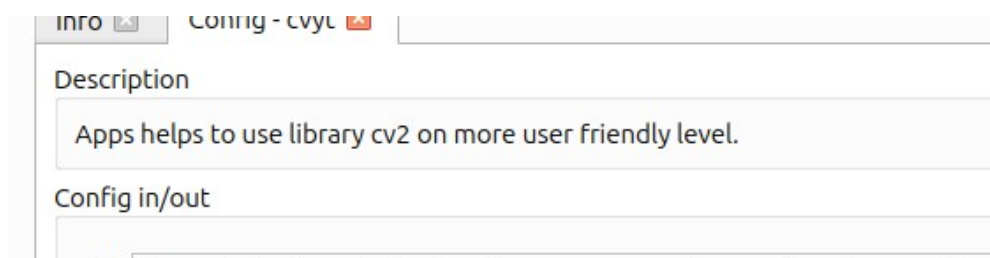
Now you are almost done. If update went well, now is time to restart the app to reload all modules.

Config

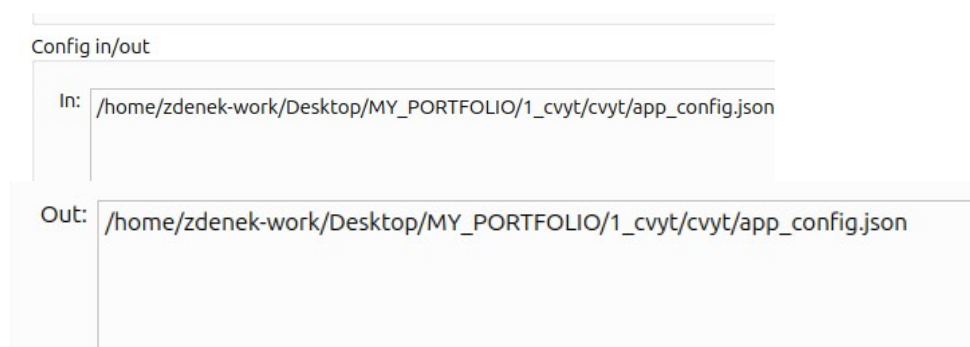
This tab is going to show you config file and gives you way how to alter it. In default is app's config file. But same functionality can be used within custom module(explanned later).

Just open the “*config tab*”(in following example for app's config).

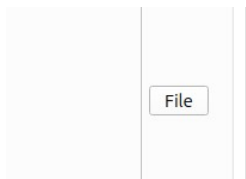
The description is taken from “*app_config.json*” file.



By default is the “*config in*” and “*config out*” set with path to file you are opening the config file.



For selection of different file use button “File”(for “Config in”). File should be loaded and presented right after you select it. In case its not valid json file, the should not be any “config tree”.



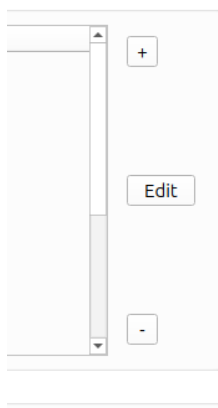
If valid config file is selected there should be visible “config tree”. Structured info from config file.

Config

Name	Value
.	.
-name	cvyt
-version	0.0.1
-title	CVYT by nohav.com
-author	nohav.com
-contact	nohav.com@gmail.com
-homep...	www.nohav.com
-descrip...	Apps helps to use library cv2 on more user friendly level.
-max_re...	==>dict
-config	==>dict
-create	==>list
-update	==>dict
-modules	==>dict
-app	==>dict

On the right side from table mentioned above are placed three buttons for operations you can apply to config file. Operations are

- add
- edit
- remove



Under the config table is placed “Config detail” area with 3 boxes.

- Name(key used in config file - json)

Config detail

1

Name:

- Value(value from config file - json)

Value:	13
--------	----

- Type(type of value – how is it stored in config file - json)

Type:	list
	dict
	string
	number

For more info about types you can use, please read chapter “*Limitations*”.

Config tree

This three shows each pair(key-value) from config file in the same format(here ist named “*Name*” + “*Value*”).

Config	
Name	Value
.	.
-name	cvyt
-version	0.0.1
-title	CVYT by nohav.co
-author	nohav.com
-contact	nohav.com@gmail
-homep...	www.nohav.com
-descrip...	Apps helps to use
-max_re...	==>dict

This means you can come across options like:

- “name” == “cvyt” ==> key = “name”, value=“cvyt”(line from dict)
- “1” == “1” ==> value = 1 or “1”, value from list
- “empty_dict” == “==>dict” ==> key = “empty_dict”, value = dict object
- “empty_list” == “==>list” ==> key = “empty_list”, value = list object
- “..1” == “==>list” ==> key = index of item in that object, value = list object

That is just tip of the iceberg. There are limitation what can be added to what, types you can use for value, etc.

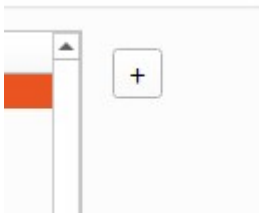
Supported type

- Int, float, string
- List, dictionary

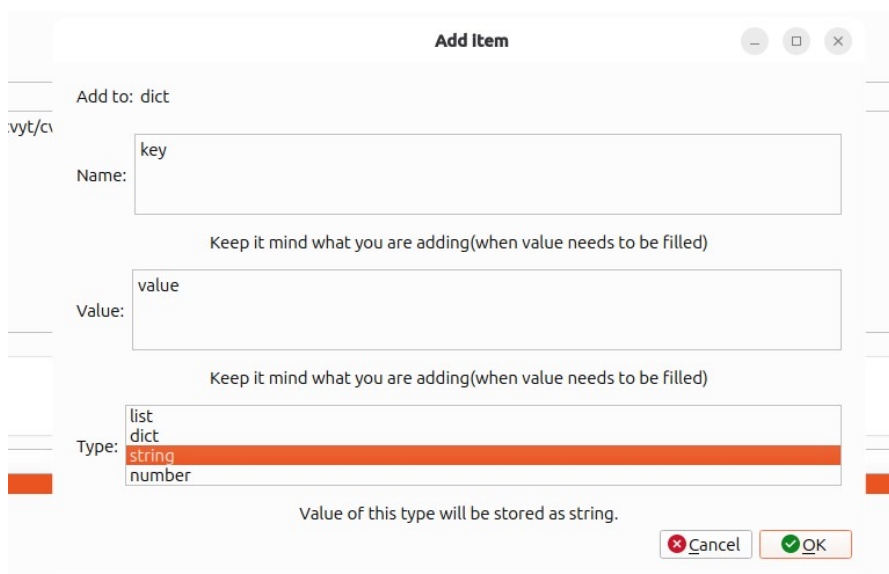
I try to cover this topic at the end of this chapter(with examples).

Add

Add new key/value to config file. Select where you want to apply this operation. For operation “Add” you needs to place your cursor to items “.” or “..”. App should eventually tell you if this operation is allowed over selected item. Next step is click to button “+”.



Next window should you give hint what to do and what can you do.

A screenshot of the 'Add Item' dialog box. It has fields for 'Add to: dict', 'Name: key', 'Value: value', and 'Type: string' (selected from a list). It includes a warning message and 'Cancel'/'OK' buttons.

Line “Add to:” tells you you are trying to ad item to what, this implies some limitation. So, now you can fill boxes “Name”, “Value”, “Type”. If you are done than click to button “Ok”.

Examples

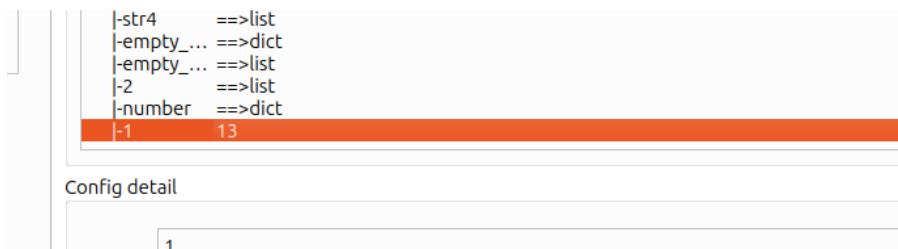
1. You want add to “dict”
 1. Fill “Name”, “Value” and “Type” of value
2. You want add to “dict” list or dict
 1. Fill “Name” and “Type” (it will takes this items only)

3. You want add to “list”
 1. Fill “Value” and “Type”(only required items)
4. You want add to “list” list or dict
 1. Fill “Type”(it will add empty list or dict)

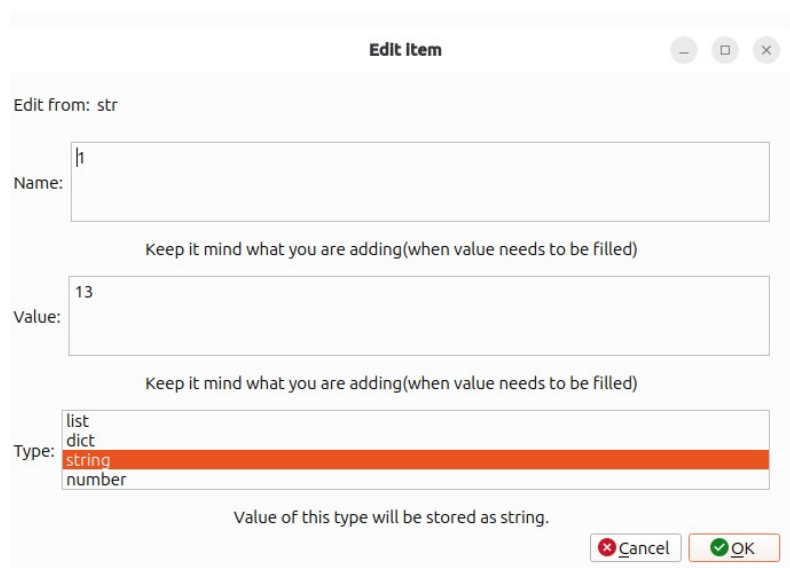
Note: Help message are shown under each box. So in case some problem you should see some help message to direct you the right way.

Edit

You need to change the value to different value or rename the key. Use button “Edit”. But before you click to the button, select what needs to be altered.



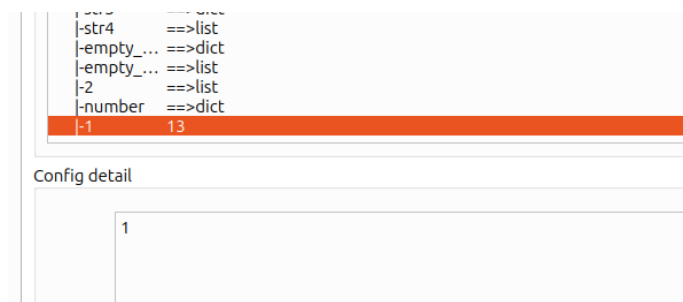
Only simple types can be edited(string, number).



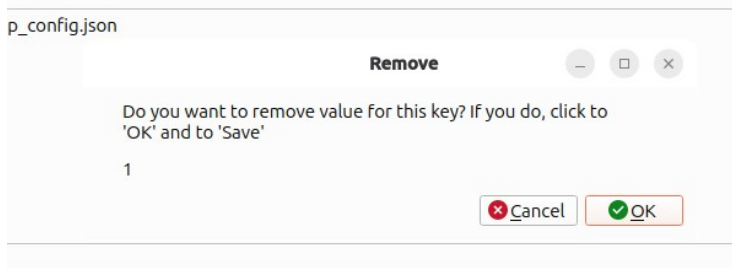
The rules/limitation are very similar as for “Add”. It means key eye on help message.

Remove

Last option is “Remove”. Select what needs to be removed.



Next step is just click to “-”. It opens know “*are you sure*” window.



Config tree, value combinations

Here I try to cover most of the possibilities/options what can you face during altering the config file.

Same logic is used for “Add”, “Edit”, “Remove” operations. As mentioned keep eye on help messages under each box.

Add

- To **list** you can add everything.
 - list ==> item is going to be added at the end of the list
 - you have specify only “Type” = list
 - dict ==> at the end of the list
 - you have specify only “Type” = dict
 - string ==> at the end of the list
 - you have to specify “Value” and “Type” = “string”
 - number(int, float) ==> at the end of the list
 - you have to specify “Value” and “Type” = “number”
- To **dict**
 - list
 - specify “Name”(key) and “Type” = “list”
 - dict
 - specify “Name”(key) and “Type” = “dict”
 - string
 - specify “Name”, “Value”, “Type” = “string”
 - number(int, float)
 - specify “Name”, “Value”, “Type” = “number”

Edit

- Number(int, float), string

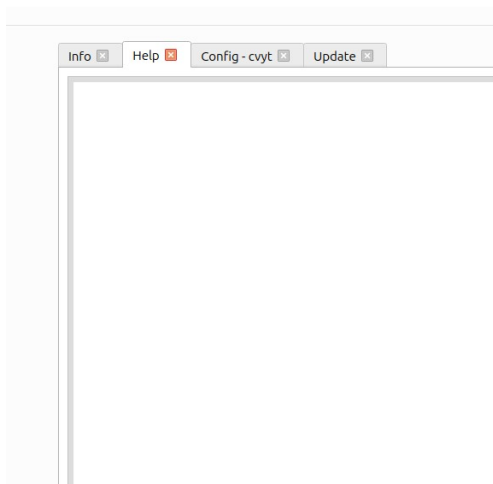
- Select the right item
- Change what needs to be changed(name, value, type)
- Other types (dict, list) can not be edited.

Remove

- Everything
 - just select(click on the item) you want remove and click remove, confirm that “*you are sure*” and its done.
 - if you are removing **dict** ==> everything in that dict is going to be removed as well
 - if you are removing **item from dict** ==> only that item(key+value) is going to be removed
 - if you are removing **item from list** ==> item, at that index will be removed(should be capable to remove item at specific index in case multiple same values in the list)
 - if you are removing **list** ==> everything from that list is going to be removed

Help tab

This tab, in case you set you want to use tab, should presents more detailed help file. Other output for operations “*double-click on Help in main menu*” is help file os opened in new window of a default browser(keep in mind it can take few seconds).



Info tab

This tab should presents basic info about the app(homepage, contact, etc.)

Info

Version: 0.0.1

Name: cvyt

Contact: nohav.com@gmail.com

Homepage: www.nohav.com

Description: Apps helps to use library cv2 on more user friendly level.

Chapter 3

Modules

Each “*logic*” is stored in folder call by unique module name e.g. “split to frames”. Every module has defined structure, which is same for each module.

Module content structure

```
- split_to_frames
  |- split_to_frames_widget.py
  |- split_to_frames_logic.py
  |- ....
  |- module_config.json
  |- help.pdf
```

Module import/start

If the model is valid and available in the list of available modules you can use it. How to import module, add it to tabs list we spoke before, but what app actually do.

Module is loaded, imported. As part of import app passes as kwargs “*main_widget*” itself, “*app_config*” object.

From “*main_widget*” you can use methods:

- *open_help_window_browser* = expecting arg
 - **help_path** ==> path to pdf file(help) to open using a default browser
- *open_config_widget* = expecting args
 - **config_path** ==> path to config file
 - **name** ==> name of the tab, should be unique(otherwise it can be mixed with existing one), e.g “*Config - filters*”

Update module

Just put together folder, put all modules(folders) to this folder and create archive from it. Open the app, click to “*Update*” tab, select the archive. Get content validated by clicking to button “*Show*”.

If content, at least some sub-folders/modules is/are valid(contains what suppose to contains) it will be shown in modules list. From this modules list you can easily select which modules you want to update and which not(checkbox). And now just “*Update*”, check if everything went well and if yes, restart the app. After the start you should see new modules or updated modules.

Chapter 4

Troubleshooting

Follow the chain of messages from UI, combined with messages from log file.

Logging

When you start this app, there should be file “*app_cvyt.log*” created. Logging file is set to be “*rotating*”, with maximum size of 1MB and keeping 3 files back to history.

Everything si broken

Good luck :)

Chapter 5

Development

Dependencies add to “*pyproject.toml*”, as package manager you can use poetry.

Tests

Basic pool of test is included. Using “*pytest*”.

Changes, new ideas, cooperation

Everyone reading this documentation, using this is welcome to join and help to develop, extend this app, so we can get maximum from it. So if you have idea, fix, suggestion or else, please don't hesitate and contact me. I'm looking for cooperation and new ideas from fresh point of view.

If you already turned your idea to the code, please create pull request. I will try to look at it as soon as possible to give you feedback.

In the block below I put some points which needs to be “DONE”, what “CAN” be done.

Future

Technical debt & New ideas

1. Polish UI(problem with flexibility of items)
2. Add overview for log file
3. Add more tests
4. Way ho to add dependencies related to modules
 1. Approach used by “Starter”(venv)
5. Create and add modules